

Generative AI & RAG

A Mini Learning Guide for Retrieval-Augmented Generation

Purpose: This document is designed as a small, self-contained knowledge base for a beginner RAG experiment. It intentionally contains specific facts, examples, and comparisons that can be tested with document-based questions.

1. What is Generative AI?

Generative AI refers to AI systems that can produce new content such as text, code, images, audio, or other outputs. Large language models (LLMs) are a major class of generative AI systems focused on language. An LLM generates responses by predicting tokens based on patterns learned during training and the context supplied at inference time.

A useful distinction is between the model's learned knowledge and information supplied at query time. A model can generate fluent answers without having access to a user's private files, internal company policies, or newly created documents.

2. The Problem RAG Solves

Retrieval-Augmented Generation (RAG) is an approach for connecting an LLM to external information. Instead of asking the model to answer only from its learned parameters, a RAG application first retrieves relevant information from a knowledge source and then provides that information to the model as context.

For example, imagine a university has a 200-page student handbook. A general LLM may not know the university's specific attendance rules. A RAG system can search the handbook for relevant passages and provide them to the LLM before it generates an answer.

Core idea: retrieve useful information first, then generate an answer using that information.

3. The Basic RAG Pipeline

A basic RAG system can be understood as two phases: an indexing phase and a query phase.

Phase	Main activities	Purpose
Indexing	Load documents; split into chunks; create embeddings; store vectors	Prepare the knowledge base for search
Query	Embed question; retrieve relevant chunks; add context; generate answer	Answer a user's question using retrieved information

4. Documents and Chunking

A document is usually too large and too unstructured to treat as one retrieval unit. Chunking means dividing a document into smaller pieces called chunks. A chunk might contain several sentences, a paragraph, a

section, or another meaningful unit.

Chunk size affects retrieval quality. If chunks are extremely small, important context may be separated. If chunks are extremely large, retrieved results may contain too much irrelevant information. A simple beginner strategy is to split text into fixed-size chunks with some overlap so neighboring chunks share a small amount of context.

Example: A report has a section called “Security Risks.” That section can be divided into several chunks. If a user asks about password risks, the retriever should ideally return the chunk containing the relevant discussion rather than the entire report.

5. Embeddings

An embedding is a numerical representation of text that captures useful aspects of its meaning. An embedding model transforms a text string into a vector: a list of numbers. Texts with related meanings can be represented by vectors that are relatively close in the embedding space.

For example, “How can I change my password?” and “I forgot my password; how do I reset it?” use different words but express a similar intent. Semantic embeddings can help a retrieval system recognize this similarity.

Embeddings are not the final answer. Their main role in a basic RAG system is to make semantic search possible.

6. Vector Search

After document chunks are embedded, the vectors can be stored in a vector store or vector database. When a user asks a question, the question is also converted into an embedding. A similarity search then looks for document vectors that are close to the question vector.

A simplified flow is: **question** → **question embedding** → **similarity search** → **relevant chunks**.

Vector search is useful because exact keyword matching is not always enough. A question may use different words from the source document while still having the same meaning.

7. Retrieval and Context

The retrieval component selects a limited set of chunks that appear relevant to the user's question. These retrieved chunks become context for the language model.

For example, if a document contains information about projects, deadlines, tools, and risks, a question about a deadline should ideally retrieve the deadline-related passages instead of unrelated sections.

Retrieval quality matters because the generator cannot reliably use information that was never retrieved. This creates an important principle: improving the LLM alone does not automatically fix a poor retrieval step.

8. Generation and Grounding

After retrieval, the application combines the user's question with the retrieved context and sends that information to an LLM. The LLM then generates a natural-language response.

Grounding means tying the answer to a specified source of information. In this mini-project, the uploaded document is the grounding source. A strong RAG instruction might tell the model to answer from the provided document and say when the document does not contain enough information.

Grounding can make answers more relevant to private or specialized information, but it does not guarantee perfect accuracy. A system can still retrieve the wrong passage or generate a claim that is not supported by the retrieved context.

9. Hallucinations in RAG

A hallucination is an answer that sounds plausible but is unsupported or incorrect. RAG is often used to reduce hallucinations by supplying source information, but retrieval does not make hallucinations impossible.

Example: Suppose a document states that a project started in 2024. If the RAG system answers that it started in 2022 without evidence in the document, that answer should be flagged.

When evaluating a RAG system, ask two separate questions: (1) Did the system retrieve the right information? and (2) Did the LLM use that information correctly?

10. RAG vs. a Plain Prompt

A plain prompt asks an LLM to answer without providing the document as a retrieved knowledge source. The model may rely on its general knowledge, the wording of the question, or assumptions.

A RAG prompt adds retrieved document context. This is especially useful when the question depends on information that is specific to the supplied document.

Approach	Main source of information	Typical risk
Plain prompt	Model's learned knowledge + prompt context	May not know document-specific facts
RAG	Retrieved source content + model capabilities	Can fail if retrieval or grounding is poor

11. Example Questions for Testing a RAG System

The following questions are intentionally document-dependent. They are better tests than generic questions because a good answer should require information from this document.

1. What are the two main phases of the basic RAG pipeline?
2. Why can chunks that are too large reduce retrieval usefulness?
3. What is the role of embeddings in a RAG system?
4. Why can vector search find relevant text when the question and document use different words?
5. What are the two separate questions that should be asked when evaluating a RAG system?
6. Does RAG guarantee that hallucinations will never occur? Explain.
7. According to this document, what is the difference between a plain prompt and a RAG approach?

12. Key Takeaways

RAG connects language models to external information. A typical pipeline loads documents, chunks them, creates embeddings, stores the vectors, retrieves relevant chunks for a query, and provides those chunks as context to an LLM. The quality of the final answer depends not only on generation, but also on the quality of document processing and retrieval.

For this learning exercise, the most important concepts are: **chunking, embeddings, vector search, retrieval, context, grounding, and hallucination evaluation.**

Note: This document is intentionally synthetic and self-contained. It is meant for a RAG learning experiment, not as a comprehensive production guide.